Lahore Urban Estates Analytics Warehouse

Project Proposal

DL3004 Data Warehousing and Business Intelligence — Final Project — Phase 1 · Version 2.1
FAST-NU Lahore · Fall 2026
Course Instructor: Mr. Muhammad Ishaq Raza · Lab Instructor: Mr. Shahzeb Mubashar
Prepared by: Junaid Minhas (23L-2559), Ahmad Waqar (23L-2634), Hanan Tiwana (23L-2536) and Eesha Khurram (23L-2641)
Date: 16 October 2026

This proposal defines the business problem, objectives, scope, data strategy and architecture of the LUE analytics warehouse. It is the entry point to the Phase 1 pack; the requirements, questions, dimensional design, grain definitions, diagrams and data dictionary are separate deliverables.

1. Executive Summary

Lahore Urban Estates (LUE) will get a PostgreSQL star-schema data warehouse that answers 14 management questions across five business processes, using 6 fact tables, 13 conformed dimensions, slowly changing dimensions and a Power BI report. Data comes from six simulated source systems in five formats and passes through Bronze, Silver and Gold layers with quality checks, a quarantine table and a full audit trail.

The design covers every fact table type taught in the course: transaction, periodic snapshot, accumulating snapshot, and factless (both event and coverage). It also uses conformed dimensions, role-playing dates, a junk dimension, degenerate dimensions, SCD Types 0, 1 and 2, inferred members for late-arriving data, and hierarchies for drill-down and roll-up.

At a glance: 5 business processes, 6 fact tables, 13 dimensions, 6 source systems, 14 business questions, 12 KPIs.

2. The Client

LUE is a fictional property management company that manages commercial plazas and developer-owned apartment blocks on behalf of private owners. It lets units, bills rent, collects payments, arranges repairs and earns a management fee on rent collected.

Portfolio: about 40 buildings in Lahore, Karachi, Islamabad and Faisalabad: residential, office and retail
Scale: about 1,800 units and 1,500 active leases; three years of history (July 2023 – June 2026)
Revenue: management fee on rent collected, service charges and late fees
Leases: 11-month residential leases; 2–5 year commercial leases
Payment channels: cash, cheque, bank transfer, Raast, JazzCash, Easypaisa
Currency and fiscal year: Pakistani Rupee (PKR); fiscal year July–June

3. Business Problem

LUE's data sits in separate systems: a property management system, Excel registers kept by different teams, bank and wallet settlement files, a tenant maintenance app and per-building leasing spreadsheets. Reporting is manual and slow, so management finds problems weeks after they start.

1. **Collections are opaque: arrears by building and ageing need manual spreadsheets.** Late payers are found only when owners complain. *Addressed by: P1 Rent invoicing and payment.*
2. **No daily view of which units are occupied.** Slow-to-let unit types are found months late. *Addressed by: P2 Unit occupancy.*
3. **Maintenance speed and vendor SLAs are not measured.** SLA breaches are anecdotal; poor vendors keep getting work. *Addressed by: P3 Maintenance.*
4. **Renewals and move-outs are not tracked.** Tenant churn by building is invisible. *Addressed by: P4 Lease lifecycle.*
5. **Incentives (e.g. one month rent-free) are offered without measuring their effect.** Discounts may cost money without shortening vacancy. *Addressed by: P5 Vacancy marketing.*
6. **Owner reports take staff several days.** Owners lose trust in LUE. *Addressed by: Cross-process (NOI per owner).*
7. **Changes overwrite old values.** Past reports cannot be reproduced. *Addressed by: SCD Type 2 dimensions.*

4. Why a Data Warehouse, Not the Operational Database

The operational systems are built to record transactions; the warehouse is built to analyse them over time. As in Lab 03, the source keeps a normalised model so updates stay consistent, while the warehouse is a dimensional model designed around business processes and analytical questions rather than copied from operational tables.

- **Purpose.** Sources record a lease, an invoice, a payment or a ticket. The warehouse analyses collections, occupancy, maintenance and owner income over time.
- **Typical question.** Source: “What is the status of invoice INV-2026-03-00831?” Warehouse: “How did the collection rate by building change over three fiscal years?”
- **History.** Sources keep only the current state. The warehouse keeps history through daily snapshots and SCD Type 2.
- **Design.** Sources are normalised (Lab 03). The warehouse uses star schemas with conformed dimensions (Labs 04–05).
- **Integration.** Each source is one system. The warehouse integrates, cleans and reconciles six.
- **Workload.** Sources handle inserts and updates. The warehouse handles aggregation, ROLLUP/CUBE, drill-down and dashboards.

5. Objectives and Success Measures

1. **Model LUE's core business processes dimensionally.** *Success measure:* 5 processes, 6 fact tables, each with one written grain.
2. **Answer management's questions.** *Success measure:* 14 questions, each traced to its fact, dimensions and OLAP operation.
3. **Preserve history where it matters.** *Success measure:* SCD Type 2 queries reproduce a past month's report exactly.
4. **Guarantee trustworthy numbers.** *Success measure:* Every planted defect is caught; source and warehouse totals reconcile on every run.

5. **Deliver usable analytics.** *Success measure:* Power BI report with 5 pages for finance, operations, owners and pipeline health.
6. **Make the pipeline reproducible.** *Success measure:* python run_pipeline.py rebuilds everything from the seeded generator.

6. Scope

In scope

- Conceptual, logical and physical dimensional models, a source ERD and an architecture diagram
- Six simulated source systems produced by a seeded generator with deliberate data defects
- A Bronze, Silver and Gold pipeline with quality checks, quarantine, audit log and reconciliation
- Star schemas with SCD Types 0, 1 and 2, special members and inferred members
- OLAP SQL (ROLLUP, CUBE, GROUPING, window functions), performance tuning and a Power BI report

Out of scope

- Real or personal data; live connections to production systems
- Real-time or streaming analytics; predictive models
- Accounting-grade general ledger and tax; co-ownership of one unit by several owners

7. Stakeholders

CEO

Needs: Portfolio health, risk
Example question: Which buildings underperform on collection and NOI?
Power BI page: Owner portfolio

Finance head

Needs: Collections, arrears, cash flow
Example question: What is this month's collection rate and arrears ageing?
Power BI page: Collections

Operations head

Needs: Vacancy, maintenance performance
Example question: Which vendors breach SLA most?
Power BI page: Maintenance

Building managers

Needs: Their own building
Example question: Who is overdue in my building? Which requests are aged?
Power BI page: Collections, Maintenance (filtered)

Leasing manager

Needs: Renewals, vacancy, incentives
Example question: Do incentives shorten vacancy?
Power BI page: Occupancy and leasing

Owner relations lead

Needs: Owner reporting
Example question: What is each owner's NOI this quarter?
Power BI page: Owner portfolio

8. Business Processes

Each business process becomes one or more fact tables. Together they cover every fact table type from Lab 05.

P1. Rent invoicing and payment

What happens: Charges are billed to leases each month; tenants pay invoice lines, often in instalments
Fact table(s): fact_rent_invoice_line, fact_payment
Fact type: Transaction (two facts)

P2. Unit occupancy

What happens: Every unit is occupied or vacant on every day, at a contract rent
Fact table(s): fact_unit_occupancy_daily
Fact type: Periodic snapshot

P3. Maintenance

What happens: A request is raised, assigned to a vendor, resolved and verified
Fact table(s): fact_maintenance_request
Fact type: Accumulating snapshot

P4. Lease lifecycle

What happens: Leases are signed, renewed, given notice and terminated
Fact table(s): fact_lease_event
Fact type: Factless (event tracking)

P5. Vacancy marketing

What happens: Incentive offers are made available on vacant units
Fact table(s): fact_vacancy_incentive_coverage
Fact type: Factless (coverage)

9. Data Strategy

All data is synthetic and generated by a seeded Python script. The generator first creates one consistent operational world, then exports each part in the format the owning team would really use. Integration, key conflicts and data quality problems are therefore genuine rather than staged.

Source systems

S1. PropertyHub (property management system)

Format: PostgreSQL schema src_pms
Contents: Buildings, units, tenants, leases, lease events, invoices, invoice lines, charge types, payments, payment allocations, payment methods
Typical defects: Duplicate payments, missing amounts, out-of-range dates, invoices before the tenant record

S2. Owner, vendor and policy registers

Format: Excel workbook (.xlsx)
Contents: Owners, unit ownership periods, vendors, SLA policy, holiday calendar
Typical defects: Inconsistent casing and spaces, typos, missing transfer dates

S3. Bank and wallet settlement files

Format: CSV, one file per channel per day
Contents: Settled transactions from bank transfer, Raast, JazzCash and Easypaisa
Typical defects: Files arriving late, references that do not match PropertyHub

S4. Tenant app maintenance tickets

Format: JSON export with nested status history
Contents: Tickets, category, priority, vendor, estimate, final cost, status changes with timestamps
Typical defects: Missing resolve times, statuses out of order

S5. Weather

Format: REST API (Open-Meteo historical, JSON)
Contents: Daily rainfall and maximum temperature for Lahore, Karachi, Islamabad and Faisalabad
Typical defects: Missing days returned as nulls

S6. Incentive log

Format: CSV exported from per-building leasing spreadsheets
Contents: Incentive offers per unit with start and end dates
Typical defects: Different date formats per building; unit codes without building prefix

Data principles

- **Operational data only.** The generator never writes warehouse tables. Every fact, including the daily occupancy snapshot, is derived by the ETL, so the transformation logic is visible and assessable.
- **Planted defects with an answer key.** Every injected defect is recorded in defects_answer_key.csv; after each run the pipeline compares its quarantine counts with this file.
- **Realism.** Rents calibrated by city and building class, seasonal move-ins, channel-specific payment delays, maintenance volume scaled by building age, and more leak tickets in monsoon months.
- **Volumes over three years.** About 1.97 million occupancy rows (1,800 units × 1,095 days), 130,000 invoice lines, 60,000 payment allocations, 8,000 lease events and 6,000 maintenance requests.

10. Architecture

The pipeline follows the 11 sections of the reference architecture diagram. Rows move from the sources through Bronze, quality checks, Silver and SCD processing into the warehouse, then to Gold marts and Power BI. Failed rows go to quarantine, and audit, performance and lineage run alongside.

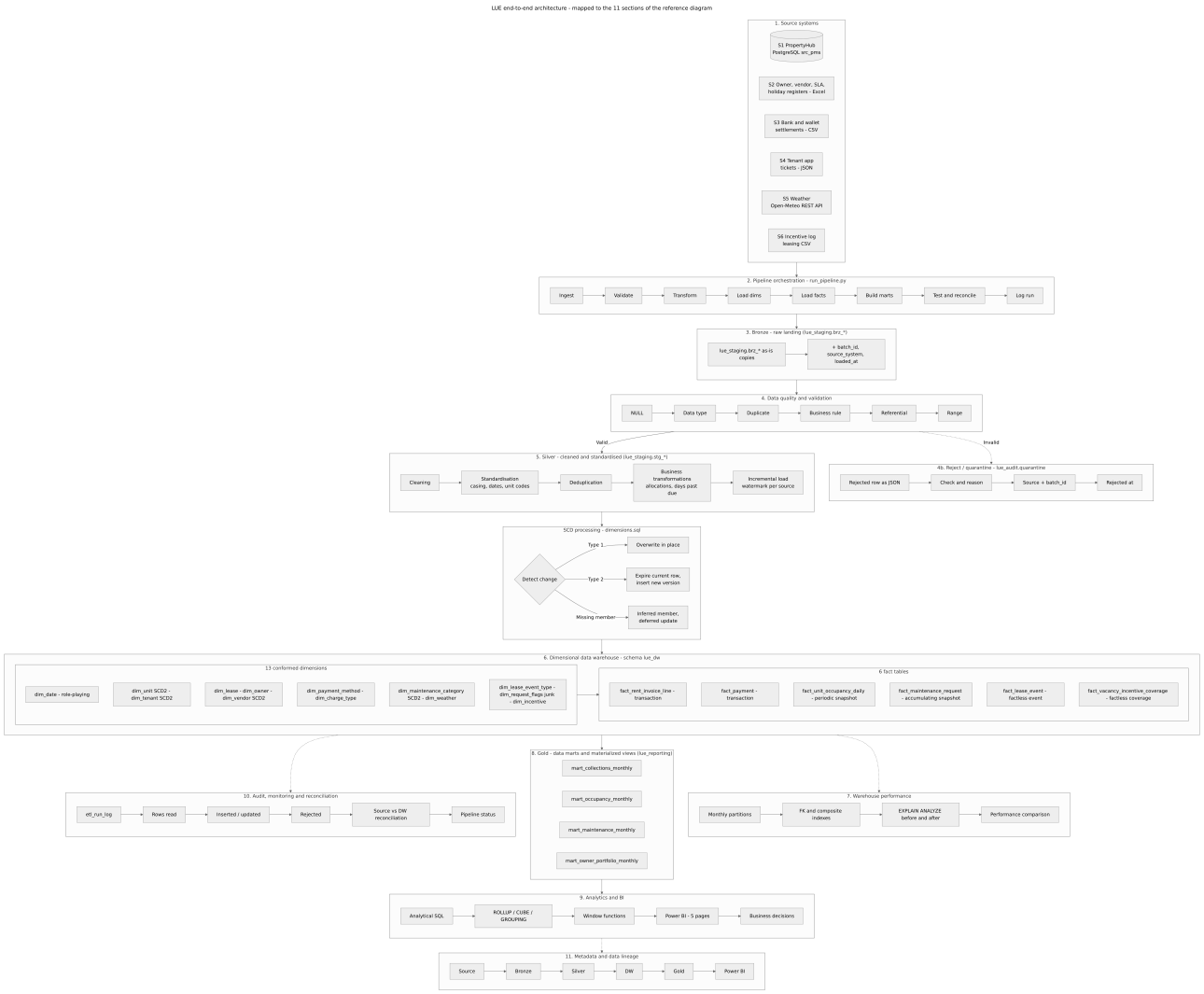


Figure 1. LUE architecture. Full-size version: 06 Schema Diagrams, Figure 17.

How each section is implemented

1. **Sources:** S1–S6, landed as files and a source schema.
2. **Orchestration:** run_pipeline.py runs every step in a fixed order with one run_id per run.
3. **Bronze (lue_staging.brz_*):** as-is copies plus batch_id, source_system and loaded_at.
4. **Quality and quarantine:** six ordered checks; failures go to lue_audit.quarantine (staging.sql).
5. **Silver (lue_staging.stg_*):** typed, standardised, deduplicated, loaded incrementally by watermark.
6. **Warehouse:** SCD processing (dimensions.sql), then 13 dimensions and 6 facts (facts.sql).
7. **Performance:** monthly partitions, indexes, EXPLAIN ANALYZE before and after.
8. **Gold (schema lue_reporting):** four marts as materialized views (Views.sql).

9. **Analytics and BI:** OLAP SQL (OLAP.sql) and the Power BI report.
10. **Audit:** run log and reconciliation results in the audit schema.
11. **Lineage:** lineage columns on every row and the lue_audit.lineage mapping.

11. Technology Stack

The build uses only the brief's free stack. Every transformation is plain SQL that can be opened and run, because the labs assess the logic, not the tooling.

- **Database:** PostgreSQL 16+ and pgAdmin 4 — *Required*. Bronze and Silver (schema lue_staging), warehouse (lue_dw), Gold (lue_reporting) and audit (lue_audit) schemas.
- **Generation and ingestion:** Python 3, pandas, numpy, Faker, requests, openpyxl — *Required*. Seeded generator; loads all six sources into Bronze; calls the weather API.
- **Transformation:** Plain SQL scripts: staging.sql, dimensions.sql, facts.sql — *Required*. Quality checks, cleaning, SCD processing, fact loads, all readable SQL.
- **Orchestration:** run_pipeline.py — *Required*. Runs every step in order and writes lue_audit.etl_run_log.
- **BI:** Power BI Desktop — *Required*. Five-page report on the Gold marts.
- **Modelling:** draw.io / dbdiagram.io / Mermaid — *Required*. Conceptual, logical and physical models, source ERD, architecture.
- **Version control:** GitHub — *Required*. Repository in the structure given in the project brief.
- **Optional:** dbt Core — *Only if time allows*. Same SQL as models; SCD written by hand, not dbt snapshot.

12. Roadmap

Phase 1 — Design (weeks 1–3, 30% of the project mark)

Work: Proposal, BRD, questions, fact/dimension identification, grain document, diagrams, data dictionary
Done when: All seven deliverables submitted and consistent word for word

Phase 2 — Implementation (weeks 4–6, 40% of the project mark)

Work: Generator, Bronze/Silver, quality checks and quarantine, SCD processing, facts, audit and reconciliation
Done when: A full run reconciles and catches every planted defect

Phase 3 — Analytics and BI (weeks 7–8, 30% of the project mark)

Work: Gold marts, OLAP SQL, performance tuning, Power BI report
Done when: Every question answered with SQL, output and a one-line interpretation

13. Phase 1 Deliverables

1. **01 Project Proposal** (PDF): Business context, objectives, scope, data strategy, architecture, roadmap. *Rubric area: Business understanding (5).*
2. **02 Business Requirements Document** (PDF): Stakeholders, business, data, quality, reporting and non-functional requirements, KPIs. *Rubric area: Requirement analysis (5).*
3. **03 Business Questions** (PDF): 14 questions, decisions supported, traceability, grain test, example SQL. *Rubric area: Requirement analysis (5).*

4. **04 Fact and Dimension Identification** (PDF): Processes, fact types, bus matrix, dimensions, hierarchies, SCD policy, keys. *Rubric area: Fact/dimension identification (10).*
5. **05 Grain Definition** (PDF): Source grains, six fact grains, measures and additivity, example rows, grain checks. *Rubric area: Fact/dimension identification (10).*
6. **06 Schema Diagrams** (PDF + PNG + Mermaid): Conceptual, logical and physical models, source ERD, hierarchies, architecture. *Rubric area: Schema design (10).*
7. **07 Data Dictionary** (Excel): Every table and column: type, keys, nullability, SCD type, source, example, rule. *Rubric area: Schema design (10).*

14. Risks and Mitigations

1. **Grain wording drifts between documents.** Impact: Inconsistent model, lost marks. Mitigation: All documents, diagrams and the dictionary are generated from one model definition.
2. **Generator takes too long.** Impact: Phase 2 slips. Mitigation: Build a small version first (2 buildings, 3 months); one member owns it from week 3.
3. **Payment allocation logic is complex.** Impact: Wrong collection rates. Mitigation: Allocate to the oldest open line first; reconcile allocated vs received totals each run.
4. **Flat synthetic data.** Impact: Dull dashboards. Mitigation: Seasonality, channel delays, monsoon leaks and planted attribute changes.
5. **Large snapshot slows Power BI.** Impact: Slow report. Mitigation: Power BI reads the Gold marts, not the 2-million-row fact.
6. **Scope creep.** Impact: Missed deadlines. Mitigation: Hold to the agreed scope; optional tools only after Phase 3 is complete.

15. Assumptions

- LUE and all data are fictional; the generator is seeded, so every run produces identical data.
- A unit has one owner at a time; ownership can change on a dated transfer recorded in the S2 register.
- A lease has one primary tenant and one unit; a renewal creates a new lease linked to the previous one.
- Rent and service charges are invoiced monthly; payments may be partial and split across invoice lines.
- LUE's management fee is assumed to be 8% of rent collected (a parameter, used in Q12).
- Fiscal year runs July to June; all amounts are in Pakistani Rupees (PKR).
- The warehouse refreshes once a day in batch; intraday latency is not required.
- Co-ownership of one unit by several owners is out of scope (it would need a bridge table).